Assignment 5 — Clustering and Initialization

Statistical Methods for Machine Learning, A.Y. 2025/26

Lorenzo Leoncini

1 Introduction

The k-means algorithm partitions a dataset into k clusters by alternating between assigning each point to its nearest centroid and recomputing centroids as cluster means. Despite its simplicity, k-means is highly sensitive to initialization: different random starting centroids can lead to widely different final clusterings. This is a concrete instance of the general challenge of empirical risk minimization in non-convex settings, the objective has many local minima, and gradient-free alternating minimization can get trapped. The k-means++ paper (Arthur & Vassilvitskii, 2007) shows that a simple randomized initialization scheme significantly improves both empirical performance and theoretical guarantees. By spreading initial centroids with probability proportional to $D(x)^2$, k-means++ demonstrates that **randomness, applied carefully, can mitigate the effects of non-convexity**: bad local minima become exponentially less likely, without requiring any prior knowledge of the true cluster structure. This assignment studies this effect by comparing the two algorithms across 50 independent trials in three settings: well-separated clusters, overlapping clusters, and high-dimensional data (MNIST).

2 Background

2.1 k-means Objective

Given $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, k-means minimizes the within-cluster sum of squares (inertia):

$$\mathcal{L} = \sum_{i=1}^{n} \min_{j \in \{1,\ldots,k\}} \|x_i - \mu_j\|^2$$

where μ_j is the centroid of cluster j. The problem is NP-hard; k-means finds a local minimum depending on initialization.

2.2 k-means++ Initialization

Each subsequent centroid is sampled with probability proportional to the squared distance from the nearest already-chosen centroid:

$$P(x \text{ chosen next}) \propto D(x)^2 = \min_{c \in C} \|x - c\|^2$$

This spreads the initial centroids out, reducing the chance of redundant or poorly placed starts.

Theoretical guarantee: k-means++ achieves an expected cost that is $O(\log k)$-competitive with the optimal clustering:

$$\mathbb{E}[\mathcal{L}] \leq 8(\ln k + 2) \cdot \mathcal{L}^*$$

No such guarantee exists for vanilla k-means.

3 Datasets

3.1 Synthetic — Well-Separated

Generated with `make_blobs`: 500 points, 5 centers, `cluster_std=1.0`, `random_state=42`. The 2D structure allows direct visualization and clear ground truth.

3.2 Synthetic — Overlapping

Same setup but `cluster_std=3.0`, creating heavily overlapping clusters. This tests the algorithm's robustness when the clustering problem is geometrically ambiguous.

3.3 MNIST Subset

First 5000 samples from MNIST (28×28 grayscale images, 784 features), normalized to $[0, 1]$. We use $k = 10$ to match the 10 digit classes. No train/test split is applied: clustering is fully unsupervised.

4 Implementation

Both algorithms are implemented from scratch using NumPy only, following the pseudocode in Arthur & Vassilvitskii (2007) and the Lloyd iteration as described in Bishop (2006), Chapter 9.1. **k-means (Lloyd's algorithm):** k points are selected uniformly at random as initial centroids. Assignment and update steps alternate until `np.allclose` convergence. An empty-cluster guard ensures that if a centroid receives no assigned points, it retains its previous position rather than producing `NaN`. **k-means++:** The first centroid is chosen uniformly at random; each subsequent centroid is sampled with probability $D(x)^2 / \sum_x D(x)^2$. Standard Lloyd iterations follow.
Inertia $\sum_i \|x_i - \mu_{\text{assign}(i)}\|^2$ is recorded at convergence. Each algorithm is run for 50 independent seeds (0 through 49).

4.1 Alternative Implementations

Several variants exist beyond the standard approach chosen here:

- **Mini-batch k-means**: updates centroids on small random subsets per iteration. Significantly faster on large datasets at a small cost in solution quality.
- **Elkan's algorithm**: exploits the triangle inequality to skip redundant distance computations, producing the same result as Lloyd's algorithm but with fewer operations.
- **Online k-means**: updates centroids one point at a time, suitable for streaming data.
- **k-means‖** (Bahmani et al., 2012): a parallelized variant of k-means++ that samples multiple candidates per round. It achieves the same $O(\log k)$ guarantee in $O(\log n)$ passes, making it suitable for distributed systems such as Apache Spark MLlib.

We use the standard Lloyd iteration with D^2-seeding as it directly reflects the paper's pseudocode and is sufficient for the scale of our experiments.

5 Experiments and Results

5.1 Overview

Each experiment runs both algorithms for 50 independent trials. For each setting we report the distribution of inertia outcomes (histogram) and a stability comparison (boxplot).

Setting	km mean	kpp mean	km variance	kpp variance
Synthetic (std=1.0)	1466.1	**1055.5**	681,836	**54,718**
MNIST (784 dims)	**194048**	194101	552,643	439,969
Overlapping (std=3.0)	**6603**	6616	31,542	34,003

Table 1: Mean and variance of inertia over 50 trials across all settings.

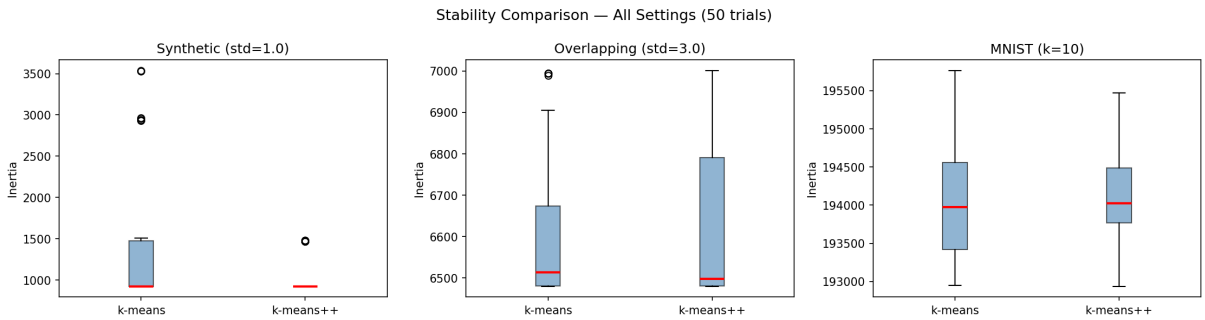

Figure 1: Boxplot of inertia distributions across 50 trials for all three settings. The difference in spread between k-means and k-means++ is immediately visible in the synthetic case and absent in the other two.

5.2 Experiment 1 — Synthetic Well-Separated Clusters

k-means++ reduces mean inertia by **28%** (1466 vs. 1055). The inertia histogram for k-means shows a heavy right tail: some runs converge to inertia above 3000, more than triple the near-optimal value, corresponding to cases where two initial centroids land in the same blob. The variance of inertia across 50 trials is 681,836 for k-means vs. 54,718 for k-means++, a **12.5× reduction**. The boxplot makes this immediately visible: k-means has a tall box with two extreme outliers at ∼3000 and ∼3500, while k-means++ produces a narrow box tightly concentrated near the optimum.

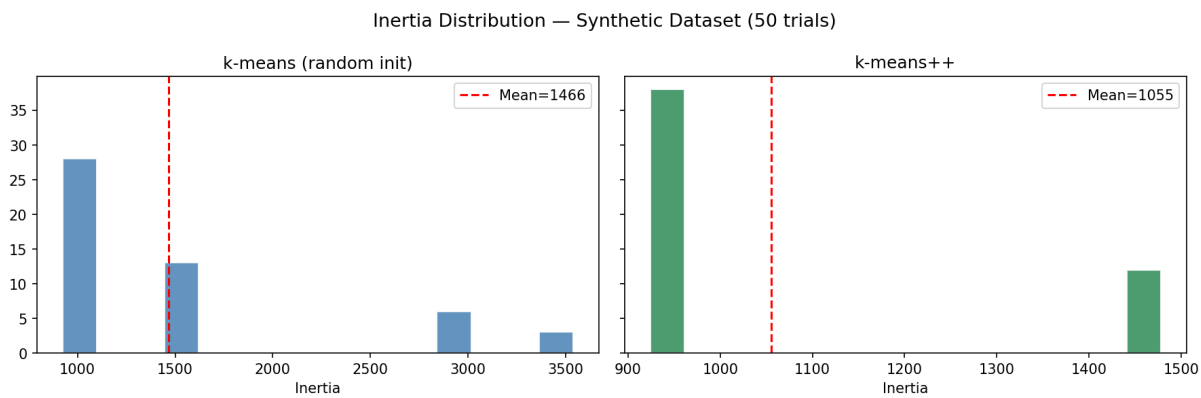

Figure 2: Inertia distribution over 50 trials on the synthetic dataset ($k = 5$, `std=1.0`). k-means (left) shows a heavy right tail with runs above 3000; k-means++ (right) concentrates tightly near the optimum.

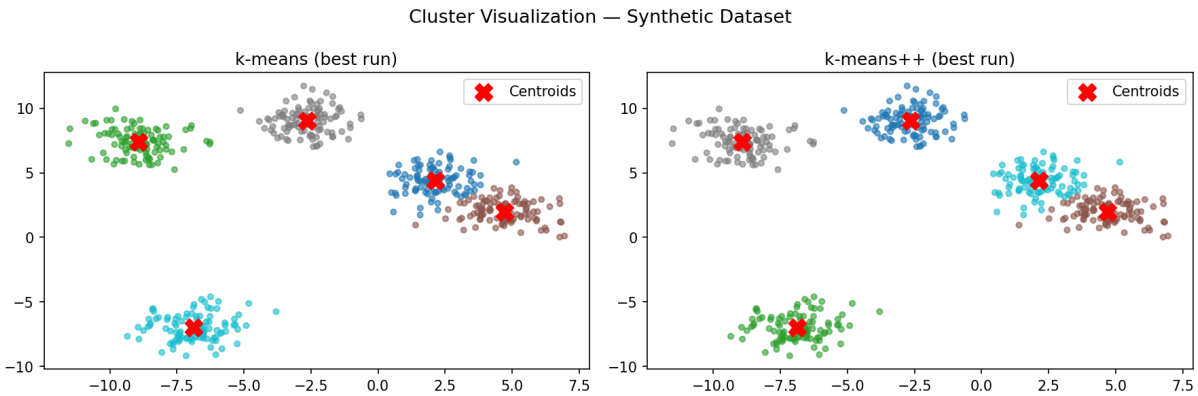

Figure 3: Best-run cluster assignments on the synthetic dataset. Both algorithms find the correct partition when initialized well; the difference lies in how often they succeed (see inertia distributions above).

5.3 Experiment 2 — MNIST (High-Dimensional) [Extension]

On 784-dimensional data, both algorithms converge to essentially the same inertia ($< 0.03\%$ difference in means). In high dimensions, pairwise distances concentrate around their mean — a consequence of the curse of dimensionality — rendering the D^2-sampling distribution nearly uniform and k-means++ equivalent to random initialization.

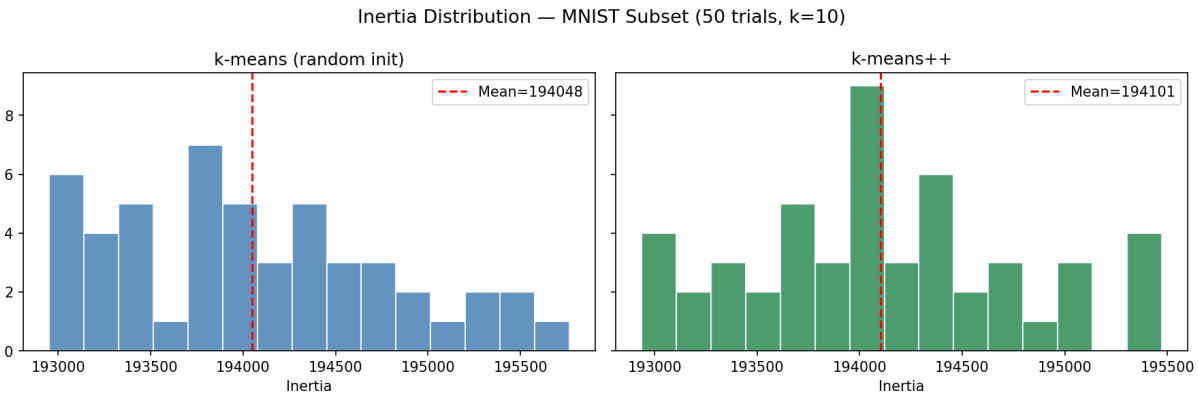

Figure 4: Inertia distribution over 50 trials on MNIST ($k = 10$, 784 dims). The two histograms are nearly identical, confirming that the k-means++ advantage vanishes in high dimensions.

5.4 Experiment 3 — Overlapping Clusters [Extension]

With `cluster_std=3.0`, the two algorithms perform nearly identically (means 6603 vs. 6616). When clusters overlap heavily, many different partitions achieve similar inertia: the objective landscape has multiple near-equivalent local minima, so the choice of initialization loses its impact. The cluster visualizations confirm this, the worst k-means run and the best k-means++ run yield qualitatively similar, ambiguous partitions.

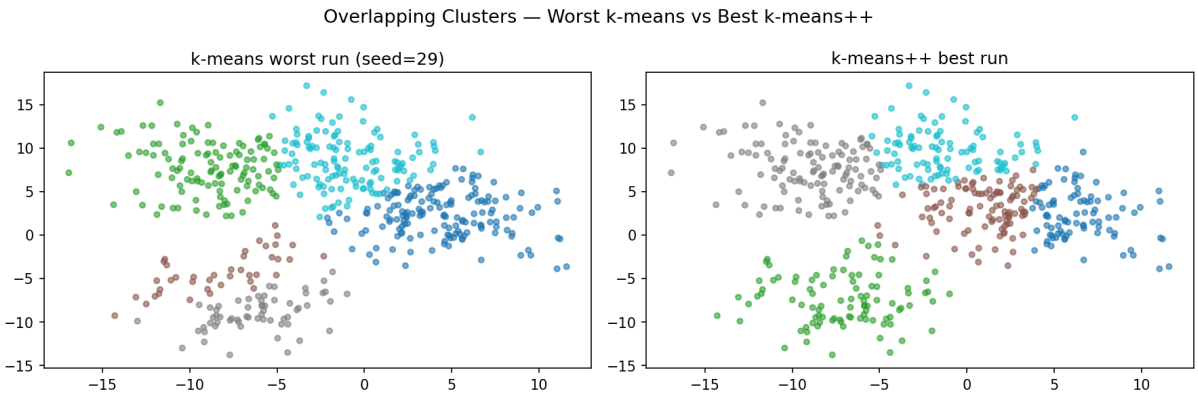

Figure 5: Worst k-means run vs. best k-means++ run on overlapping clusters (`std=3.0`). The partitions are visually indistinguishable, confirming that initialization strategy is irrelevant when cluster geometry is ambiguous.

6 Discussion

The three experiments reveal a consistent pattern: **the advantage of k-means++ is tied to clear cluster geometry**. In the well-separated 2D setting, D^2-sampling works as intended: spreading centroids across distinct blobs dramatically reduces variance and mean inertia, confirming the $O(\log k)$ theoretical guarantee in practice. In the overlapping setting, the problem itself becomes ambiguous: there is no single correct partition, and many configurations have similar inertia. In this regime, initialization strategy is irrelevant because the algorithm finds a reasonable local minimum regardless. In high dimensions, the curse of dimensionality makes all distances nearly equal, collapsing the D^2-sampling to near-uniform. The k-means++ advantage disappears not because the algorithm fails, but because the geometric intuition behind it no longer applies. The takeaway: k-means++ helps most when k is large relative to the spread of data, clusters are well-separated, and dimensionality is low to moderate.

Note on methodology: the original paper reports inertia as a ratio to the optimal solution $\mathcal{L}^*$, which is NP-hard to compute exactly. We compare the two algorithms directly against each other, which is sufficient to validate the empirical advantage of k-means++ without requiring an intractable reference solution.

7 Conclusion

We implemented k-means and k-means++ from scratch and compared them over 50 trials across three settings. On well-separated synthetic data, k-means++ is clearly superior in both quality and stability, validating the $O(\log k)$ theoretical guarantee. On overlapping clusters and high-dimensional MNIST data, the advantage disappears for different but related reasons, geometric ambiguity and distance concentration, respectively. Initialization matters most when the problem has a clear structure to exploit.

References

- Arthur, D. & Vassilvitskii, S. (2007). *k-means++: The Advantages of Careful Seeding.* SODA 2007.

- Lloyd, S.P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.
- Bishop, C.M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 9.1.
- Bahmani, B. et al. (2012). Scalable k-means++. *Proceedings of the VLDB Endowment*, 5(7), 622–633.

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.